The RLS Adaptive Filter: Array and Fast Array Algorithms

Notes, ECE 6030, Spring 2022

Todd K. Moon

These notes were drawn and modified from *Fundamentals of Adaptive Filtering* by Ali H. Sayed, (Wiley-Interscience 2003)

1 Toward a QR-oriented approach: Unitary matrices, norm and angle preservation

Let $\mathbf{x}$ and $\mathbf{y}$ be vectors that are related by a unitary matrix Θ,

$$\Theta\mathbf{x} = \mathbf{y}.$$

Then, as we know, the norm is preserved:

$$\|\mathbf{y}\|^2 = \mathbf{y}^H\mathbf{y} = \mathbf{x}^H\Theta^J\Theta\mathbf{x} = \mathbf{x}^H\mathbf{x} = \|\mathbf{x}\|^2.$$

If we also have vectors $\mathbf{a}$ and $\mathbf{b}$ that are related by that same Θ,

$$\Theta\mathbf{a} = \mathbf{b}$$

then inner products are also preserved:

$$\mathbf{y}^H\mathbf{b} = \mathbf{x}^H\Theta^H\Theta\mathbf{a} = \mathbf{x}^H\mathbf{a}.$$

This is referred to as the angle preservation property.

The norm preservation can actually be stated somewhat more strongly than the result above just indicated, since the converse is true as well.

Lemma 1 *(Norm preservation lemma) Let A and B be two $M \times n$ matrices, with $n \leq M$. Then $A^HA = B^HB$ if and only there exists a $M \times M$ unitary matrix Θ such that $A = \Theta B$.*

The forward direction is straightforward: If $A = \Theta B$, then

$$A^HA = B^H\Theta^H\Theta = B^HB$$

by the unitary property of Θ. The converse is most easily done using the SVD (which we haven't met yet).

Now consider the following problem: Given scalars a, b, d, and e, we desire to simultaneously solve the equations

$$|a|^2 + |b|^2 = |c|^2 \quad a^*d + b^*e = c^*f$$

One method of solution is obvious: simply compute $|c|^2$ in the first equation, take a "square root" to find c^*, then solve for f in the second equation. We present a more circuitous, but more helpful route.

We note that we can write these equations in a vector form:

$$\begin{bmatrix} a^* & b^* \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} c^* & 0 \end{bmatrix} \begin{bmatrix} c \\ 0 \end{bmatrix}$$
$$\begin{bmatrix} a^* & b^* \end{bmatrix} \begin{bmatrix} d \\ e \end{bmatrix} = \begin{bmatrix} c^* & 0 \end{bmatrix} \begin{bmatrix} \mathsf{y} \\ \mathsf{z} \end{bmatrix}$$

where $\mathsf{y} = f$.

We see that the first equation is a instance of norm preservation. By the norm preservation lemma, there is a unitary matrix Θ such that

$$\Theta \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} c \\ 0 \end{bmatrix}.$$

In fact, we have seen that the matrix Θ must be a Givens rotation. For complex a and b, it can be expressed as

$$\Theta = \begin{cases} \frac{1}{\sqrt{1+|\rho|^2}} \begin{bmatrix} 1 & \rho^* \\ -\rho & 1 \end{bmatrix} & \text{if } a \neq 0, \text{ with } \rho = b/a \\ \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} & \text{if } a = 0. \end{cases}$$

(or any unit-magnitude scalar complex multiple of these). Note: If it desired to have the rotated value to be real — as it often is, choose

$$\Theta = \begin{cases} \frac{e^{-j\phi_a}}{\sqrt{1+|\rho|^2}} \begin{bmatrix} 1 & \rho^* \\ -\rho & 1 \end{bmatrix} & \text{if } a \neq 0, \text{ with } \rho = b/a \\ e^{-j\phi_b} \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} & \text{if } a = 0. \end{cases}$$

where $a = |a|e^{j\phi_a}$ and $b = |b|e^{j\phi_b}$.

When we multiply $\left[\begin{smallmatrix} a \\ b \end{smallmatrix}\right]$ by Θ, the result is that the "energy" in the vector is concentrated in the first element,

$$\Theta \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} \mathsf{x} \\ 0 \end{bmatrix},$$

where $\mathsf{x} = \sqrt{|a|^2 + |b|^2}$. If we "square" both sides of the equation we get

$$\begin{bmatrix} a^* & b^* \end{bmatrix} \Theta^H \Theta \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} \mathsf{x}^* & 0 \end{bmatrix} \begin{bmatrix} \mathsf{x} \\ 0 \end{bmatrix}$$

or $|a|^2 + |b|^2 = |\mathsf{x}|^2$, from which $\mathsf{x} = c$.

Now let us look at the second equation. By the angle preservation property, since we have

$$\Theta \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} c \\ 0 \end{bmatrix}$$

if we take

$$\Theta \begin{bmatrix} d \\ e \end{bmatrix} = \begin{bmatrix} \mathsf{y} \\ \mathsf{z} \end{bmatrix}$$

then we must have

$$\begin{bmatrix} a^* & b^* \end{bmatrix} \begin{bmatrix} d \\ e \end{bmatrix} = \begin{bmatrix} c^* & 0 \end{bmatrix} \begin{bmatrix} \mathsf{y} \\ \mathsf{z} \end{bmatrix} = c^* \mathsf{y}.$$

from which we recognize that we must have $\mathsf{y} = f$. It must furthermore be the case that $\left[\begin{smallmatrix} d \\ e \end{smallmatrix}\right]$ and $\left[\begin{smallmatrix} \mathsf{y} \\ \mathsf{z} \end{smallmatrix}\right]$ have the same Euclidean norms, so that

$$|\mathsf{z}|^2 + |f|^2 = |d|^2 + |e|^2.$$

It will be helpful to consider this as an array. Form the *pre-array*

$$\mathcal{A} = \begin{bmatrix} a & d \\ b & e \end{bmatrix}$$

and choose a unitary matrix Θ which (upper) triangularizes $\mathcal{A}$, as

$$\Theta \begin{bmatrix} a & d \\ b & e \end{bmatrix} = \begin{bmatrix} \mathsf{x} & \mathsf{y} \\ 0 & \mathsf{z} \end{bmatrix}$$

We observe that the Θ matrix is determined only by the first column of $\mathcal{A}$, and that, as before, Θ is a Givens rotation matrix. Now "square" both sides of the equation:

$$\mathcal{A}^H \mathcal{A} = \begin{bmatrix} a^* & b^* \\ d^* & e^* \end{bmatrix} \begin{bmatrix} a & d \\ b & e \end{bmatrix} = \mathcal{A}^H \underbrace{\Theta^H \Theta}_{I} \mathcal{A} = \begin{bmatrix} \mathsf{x}^* & 0 \\ \mathsf{y}^* & \mathsf{z}^* \end{bmatrix} \begin{bmatrix} \mathsf{x} & \mathsf{y} \\ 0 & \mathsf{z} \end{bmatrix}$$

Equating terms on both sides gives the relations

$$|a|^2 + |b|^2 = |\mathsf{x}|^2 \qquad \mathsf{x}^*\mathsf{y} = a^*d + b^*e$$

from which we identify $\mathsf{x} = c$ and $\mathsf{y} = f$.

1.1 The matrix case

We now extend the scalar norm/angle preservation problem to the matrix case. Let A, B, D and E be known matrices (or vectors), and consider the problem of determining an (upper) triangular matrix C and a matrix F in the equations

$$\begin{aligned} C^HC &= A^HA + B^HB \\ C^HF &= A^HD + B^HE \qquad \text{or} \qquad F^HC = D^HA + E^HB \end{aligned} \tag{1}$$

We proceed as before, forming the pre-array

$$\mathcal{A} = \begin{bmatrix} A & D \\ B & E \end{bmatrix}$$

We seek a unitary matrix Θ which (upper) triangularizes $\mathcal{A}$:

$$\Theta\mathcal{A} = \Theta \begin{bmatrix} A & D \\ B & E \end{bmatrix} = \begin{bmatrix} \mathsf{X} & \mathsf{Y} \\ \mathbf{0} & \mathsf{Z} \end{bmatrix}.$$

where X is upper triangular, and (depending on the choice made for Θ) Z is also. Now "square" this equation:

$$\mathcal{A}^H\mathcal{A} = \mathcal{A}^H \underbrace{\Theta^H\Theta}_{I}\mathcal{A} = \begin{bmatrix} \mathsf{X}^H & \mathbf{0} \\ \mathsf{Y}^H & \mathsf{Z} \end{bmatrix} \begin{bmatrix} \mathsf{X} & \mathsf{Y} \\ \mathbf{0} & \mathsf{Z} \end{bmatrix} = \begin{bmatrix} \mathsf{X}^H\mathsf{X} & \mathsf{X}^H\mathsf{Y} \\ \mathsf{Y}^H\mathsf{X} & \mathsf{Y}^H\mathsf{Y} + \mathsf{Z}^H\mathsf{Z} \end{bmatrix}$$

Equating corresponding blocks, we obtain

$$\begin{aligned} A^HA + B^HB &= \mathsf{X}^H\mathsf{X} \\ A^HD + B^HE = \mathsf{X}^H\mathsf{Y} \qquad &\text{or} \qquad D^HA + E^HB = \mathsf{Y}^H\mathsf{X} \end{aligned}$$

from which we identify

$$C = \mathsf{X} \qquad F = \mathsf{Y}.$$

2 Weighted Regularized Least Squares

We will start with an optimization problem:

$$\min_{\mathbf{w}} \left[\underbrace{(\mathbf{w} - \overline{\mathbf{w}})^H\Pi(\mathbf{w} - \overline{\mathbf{w}})}_{\text{regularizer}} + \underbrace{(\mathbf{y} - H\mathbf{w})^HW(\mathbf{y} - H\mathbf{w})}_{\text{squared error}} \right]$$

where W and Π are symmetric positive definite. As indicated, the first term is a *regularizer*. The encourages the solution $\mathbf{w}$ to be close to the nominal solution $\overline{\mathbf{w}}$. It also helps make the numerics better and stabilizes matrix inversion. The solution is

$$\hat{\mathbf{w}} = \overline{\mathbf{w}} + [\Pi + H^HWH]^{-1}H^HW(\mathbf{y} - H\overline{\mathbf{w}})$$

and the resulting minimum cost is

$$\xi = (\mathbf{y} - H\overline{\mathbf{w}})^HW\tilde{\mathbf{y}} = (\mathbf{y} - H\overline{\mathbf{w}})^H[W^{-1} + H\Pi^{-1}H^H]^{-1}(\mathbf{y} - H\overline{\mathbf{w}}) \tag{2}$$

where $\tilde{\mathbf{y}} = \mathbf{y} - H\hat{\mathbf{w}}$. There is an orthogonality condition satisfied by $\hat{\mathbf{w}}$:

$$H^HW\tilde{\mathbf{y}} = \Pi(\hat{\mathbf{w}} - \overline{\mathbf{w}})$$

or

$$(H^HWH + \Pi)\hat{\mathbf{w}} = H^HW\mathbf{y} + \Pi\overline{\mathbf{w}}.$$

3 The RLS Adaptive Filter Algorithm

We review the RLS algorithm, in somewhat more generality than presented in the book. Consider a collection of data $\{\mathbf{u}_j, d(j)\}_{j=0}^{N-1}$, where the $\mathbf{u}_j$ are $M \times 1$ vectors called *regressors* and the $d(j)$ are scalars representing *desired values*. We observe that the regressors can be any sequence of vectors, not necessarily having a time delay structure as we have previously talked about. (Array processing.) Notation: Following Sayed, scalar values are indexed with parenthesis, as in $u(i)$, while vectors are indexed with subscripts, as in $\mathbf{u}_i$. We also introduce a symmetric positive-definite matrix Π and a scalar λ such that $\lambda < 1$ is close to 1. Let $\mathbf{w}_{N-1}$ denote the vector of FIR filter coefficients at time $N-1$. The output of the filter at due to the regressor at time j is $\mathbf{u}_j^H \mathbf{w}_j^H$. We desire to find filter coefficients $\mathbf{w}$ such that the weighted cumulative squared error

$$\sum_{j=0}^{N-1} \lambda^{N-1-j} |d(j) - \mathbf{u}_j^H \mathbf{w}|^2$$

is minimized. In this formulation, we also introduce a *regularizer*. let $\overline{\mathbf{w}}$ denote some nominal value. The deviation of $\mathbf{w}$ from $\overline{\mathbf{w}}$ is $\lambda^{N-1}(\mathbf{w} - \overline{\mathbf{w}})^H \Pi (\mathbf{w} - \overline{\mathbf{w}})$. The factor λ^{N-1} indicates that as time progresses, the deviation from the initial $\overline{\mathbf{w}}$ becomes less significant. The RLS filter computes the solution

$$\mathbf{w}_{N-1} = \arg\min_{\mathbf{w}} \left[\underbrace{\lambda^N (\mathbf{w} - \overline{\mathbf{w}})^H \Pi (\mathbf{w} - \overline{\mathbf{w}})}_{\text{regularizer}} + \underbrace{\sum_{j=0}^{N} \lambda^{N-1-j} |d(j) - \mathbf{u}_j^H \mathbf{w}|^2}_{\text{squared error}} \right] \tag{3}$$

which minimizes the total, regularized, weighted cost using inputs up to an including time $N-1$. We are also interested in $\xi(j)$, the corresponding minimum cost. We will find it more convenient to set $\overline{\mathbf{w}} = \mathbf{0}$. If we desire to use some $\overline{\mathbf{w}} \neq 0$, this can be achieved simply by setting $\mathbf{w}_{-1} = \overline{\mathbf{w}}$, and using the RLS algorithm as described below.

It is convenient to express the problem (11) in a matrix/vector notation. Let

$$\mathbf{y}_{N-1} = \begin{bmatrix} d(0) \\ d(1) \\ \vdots \\ d(N-1) \end{bmatrix} \qquad H_{N-1} = \begin{bmatrix} \mathbf{u}_0^H \\ \mathbf{u}_1^H \\ \vdots \\ \mathbf{u}_{N-1}^H \end{bmatrix}.$$

Note that there is a straightforward time update for this:

$$\mathbf{y}_N = \begin{bmatrix} \mathbf{y}_{N-1} \\ d(N) \end{bmatrix} \qquad H_N = \begin{bmatrix} H_{N-1} \\ \mathbf{u}_N^H \end{bmatrix}.$$

Also, let

$$\Lambda_{N-1} = \text{diag}(\lambda^{N-1}, \lambda^{N-2}, \ldots, \lambda, 1).$$

Then the weighted, regularized problem can be written as

$$\mathbf{w}_{N-1} = \min_{\mathbf{w}} \left[\lambda^N \mathbf{w}^H \Pi \mathbf{w} + (\mathbf{y}_{N-1} - H_{n-1}\mathbf{w})^H \Lambda_{N-1} (\mathbf{y}_{N-1} - H_{N-1}\mathbf{w}) \right]$$

We make the identification with the weighted regularized least squares problem

$$\lambda^N \Pi \to \Pi \qquad \Lambda_{N-1} \to W.$$

The solution is found from the orthogonality condition

$$\left[\lambda^N \Pi + H_{N-1}^H \Lambda_{N-1} H_{N-1} \right] \mathbf{w}_{N-1} = H_{N-1}^H \Lambda_{N-1} \mathbf{y}_{N-1}$$

We now define:

$$P_{N-1} = \left[\lambda^N \Pi + H_{N-1}^H \Lambda_{N-1} H_{N-1} \right]^{-1}$$

We thus have the solution

$$\mathbf{w}_{N-1} = P_{N-1} H_{N-1}^H \Lambda_{N-1} \mathbf{y}_{N-1} \tag{4}$$

If we move to the next time instant, we similarly obtain

$$\mathbf{w}_N = P_N H_N^H \Lambda_N \mathbf{y}_{N-1} = P_N(\lambda H_{N-1}^H \Lambda_{N-1} \mathbf{y}_{N-1} + \mathbf{u}_N^H d(N)).$$

where

$$P_N = \left[\lambda^{N+1}\Pi + H_N^H \Lambda_N H_N\right]^{-1} \tag{5}$$

We will develop an efficient update for this. Note that

$$P_N^{-1} = \lambda^{N+1}\Pi + H_N^H \Lambda_N H_N = \lambda\lambda^N \Pi + \lambda H_{N-1}^H \Lambda_{N-1} H_{N-1} + \mathbf{u}_N \mathbf{u}_N^H \lambda P_{N-1}^{-1} + \mathbf{u}_N \mathbf{u}_N^H = \lambda(P_{N-1} + \lambda^{-1}\mathbf{u}_N \mathbf{u}_N^H).$$

Now employ the matrix inversion lemma to obtain

$$P_N = \frac{1}{\lambda}(P_{N-1} - \frac{\lambda^{-1} P_{N-1}\mathbf{u}_N \mathbf{u}_N^H P_{N-1}}{1 + \lambda^{-1}\mathbf{u}_N^H P_{N-1}\mathbf{u}_N}) = \lambda^{-1}P_{N-1} - \frac{\lambda^{-2} P_{N-1}\mathbf{u}_N \mathbf{u}_N^H P_{N-1}}{1 + \lambda^{-1}\mathbf{u}_N^H P_{N-1}\mathbf{u}_N} \tag{6}$$

So

$$\begin{aligned}\mathbf{w}_N &= (\lambda^{-1}P_{N-1}^{-1} - \frac{\lambda^{-2} P_{N-1}\mathbf{u}_N \mathbf{u}_N^H P_{N-1}}{1 + \lambda^{-1}\mathbf{u}_N^H P_{N-1}\mathbf{u}_N})(\lambda H_{N-1}^H \Lambda_{N-1}\mathbf{y}_{N-1} + \mathbf{u}_N d(N)) \\ &= \mathbf{w}_{N-1} + \frac{\lambda^{-1}P_{N-1}\mathbf{u}_N}{1 + \lambda^{-1}\mathbf{u}_N^H P_{N-1}\mathbf{u}_N}[d(N) - \mathbf{u}_N^H \mathbf{w}_{N-1}]\end{aligned}$$

We define the *conversion factor* by

$$\gamma(N) = \frac{1}{1 + \lambda^{-1}\mathbf{u}_N^H P_{N-1}\mathbf{u}_N}$$

and the *gain vector*

$$\mathbf{g}_N = \lambda^{-1}P_{N-1}\mathbf{u}_N \gamma(N).$$

Expressed in terms of these quantities, the weight update is

$$\mathbf{w}_N = \mathbf{w}_{N-1} + \mathbf{g}_N(d(N) - \mathbf{u}_N^H \mathbf{w}_{N-1}).$$

We also have

$$P_N = \lambda^{-1}P_{N-1} - \mathbf{g}_N \mathbf{g}_N^H / \gamma(i).$$

Some other forms of the conversion factor and the gain vector are also important. Multiplying (6) on both sides by $\mathbf{u}_N$ we have

$$\begin{aligned}P_N\mathbf{u}_N &= \lambda^{-1}P_{N-1}\mathbf{u}_N - \frac{\lambda^{-2} P_{N-1}\mathbf{u}_N \mathbf{u}_N^H P_{N-1}\mathbf{u}}{1 + \lambda^{-1}\mathbf{u}_N^H P_{N-1}\mathbf{u}_N} \\ &= \frac{\lambda^{-1}P_{N-1}\mathbf{u}_N}{1 + \lambda^{-1}\mathbf{u}_N^H P_{N-1}\mathbf{u}_N} \qquad \text{(common denominator; simplify)} \\ &= \mathbf{g}_N.\end{aligned}$$

Now multiply both sides on the left by $\mathbf{u}_N^H$:

$$\mathbf{u}_N^H P_N \mathbf{u}_N = \frac{\lambda^{-1}\mathbf{u}_N^H P_{N-1}\mathbf{u}_N}{1 + \lambda^{-1}\mathbf{u}_N^H P_{N-1}\mathbf{u}_N}.$$

Now subtract both sides from 1:

$$\begin{aligned}1 - \mathbf{u}_N^H P_N \mathbf{u}_N &= 1 - \frac{\lambda^{-1}\mathbf{u}_N^H P_{N-1}\mathbf{u}_N}{1 + \lambda^{-1}\mathbf{u}_N^H P_{N-1}\mathbf{u}_N} \\ &= \frac{1}{1 + \lambda^{-1}\mathbf{u}_N^H P_{N-1}\mathbf{u}_N} \qquad \text{(common denominator; simplify)} \\ &= \gamma(N)\end{aligned}$$

Hence we have the equivalent ways of expressing:

$$\gamma(N) = 1 - \mathbf{u}_N^H P_N \mathbf{u}_N$$

and

$$\mathbf{g}_N = P_n \mathbf{u}_N.$$

Combining these we obtain

$$\gamma(N) = 1 - \mathbf{u}_N^H \mathbf{g}_N. \tag{7}$$

Associated with the RLS adaptive filter algorithm are two error quantities. The first is the *a priori* output error, $e(N)$. This is the error due to the filter coefficients before they are updated:

$$e(N) = d(N) - \mathbf{u}_N^H \mathbf{w}_{N-1}$$

The *a posteriori* output error $r(N)$ is the error after the coefficients are updated:

$$r(N) = d(N) - \mathbf{u}_N^H \mathbf{w}_N.$$

Now we observe that

$$\begin{aligned} r(N) &= d(N) - \mathbf{u}_N^H(\mathbf{w}_{N-1} + \mathbf{g}_N[d(N) - \mathbf{u}_N^H \mathbf{w}_{N-1}]) \\ &= d(N) - \mathbf{u}_N^H \mathbf{w}_{N-1} - \mathbf{u}_N^H \mathbf{g}_N e(N) \\ &= e(N) - \mathbf{u}_N^H \mathbf{g}_N e(N) = (1 - \mathbf{u}_N^H \mathbf{g}_N) e(N). \end{aligned}$$

In light of (7) we have

$$r(N) = \gamma(N) e(N).$$

Hence, the factor $\gamma(N)$ *converts* the *a priori* error to the *a posteriori* error. Note that by the definition of $\gamma(N)$,

$$0 \le \gamma(N) \le 1$$

so that

$$|r(N)| \le |e(N)|.$$

That is, the adaptation never makes the instantaneous error larger.

While not needed for the actual adaptive filter, it is generally helpful to track the minium residual squared error. Let $\xi(N-1)$ denote the minimum cost of the optimization problem (11). From (2) we have

$$\xi_N = \mathbf{y}_N^H \Lambda_N (\mathbf{y} - H_N \mathbf{w}_N)$$

and

$$\xi_{N-1} = \mathbf{y}_{N-1}^H \Lambda_{N-1} (\mathbf{y}_{N-1} - H_{N-1} \mathbf{w}_{N-1}).$$

We want to develop a recursive formula for computing ξ_N. We can write

$$\begin{aligned} \xi(N) &= \begin{bmatrix} \mathbf{y}_{N-1}^H & d^*(N) \end{bmatrix} \Lambda_N \begin{bmatrix} \mathbf{y}_{N-1} - H_{N-1}(\mathbf{w}_{N-1} + \mathbf{g}_N e(N)) \\ d(N) - \mathbf{u}_N^H(\mathbf{w}_{N-1} + \mathbf{g}_N e(N) \end{bmatrix} \\ &= \begin{bmatrix} \mathbf{y}_{N-1}^H & d^*(N) \end{bmatrix} \Lambda_N \begin{bmatrix} \mathbf{y}_{N-1} - H_{N-1}\mathbf{w}_{N-1} - H_{N-1}\mathbf{g}_N e(N) \\ e(N) \underbrace{(1 - \mathbf{w}_N^H \mathbf{g}_N)}_{\gamma_N} \end{bmatrix} \\ &= \underbrace{\lambda \mathbf{y}_{N-1}^H \Lambda_{N-1}(\mathbf{y}_{N-1} - H_{N-1}\mathbf{w}_{N-1})}_{\lambda\xi(N-1)} - \mathbf{y}_{N-1}^H \Lambda_N H_{N-1} \underbrace{\mathbf{g}_N}_{P_{N-1}\mathbf{u}_N\gamma(N)} e(N) + d^*(N)\Lambda_N e(N)\gamma(N) \\ &= \lambda\xi(N-1) + e(N)\gamma(N)[d^*(N) - \underbrace{\mathbf{y}_{N-1}^H \Lambda_N H_{N-1} P_{N-1}}_{\mathbf{w}_{N-1}^H \text{ from (4)}} \mathbf{u}_N \\ &= \lambda\xi(N-1) + |e(N)|^2 \gamma(N) \\ &= \lambda\xi(N-1) + |r(N)|^2/\gamma(N) \end{aligned}$$

where the last equality follows since

$$|e(N)|^2\gamma(N) = |r(N)/\gamma(N)|^2\gamma(N).$$

In the recursive algorithm, computing P_N using P_{N-1} using the matrix inversion lemma, there is always a question of how to get the whole process started: How is P_{-1} to be rationally selected? From (5), we have $P_N = \left[\lambda^{N+1}\Pi + H_N^H \Lambda_N H_N\right]^{-1}$. When $N = -1$, the H_N matrix is empty, and we obtain

$$P_{-1} = \Pi^{-1}.$$

This selection provides a consistent way of getting the recursion started, and indicates that the initial value actually acts as a regularizer of the least-squares problem. Usually, Π is chosen to be proportional to the identity matrix.

Putting all the pieces together, the RLS algorithm can be expressed as follows:

Start with $\mathbf{w}_{-1} = 0$ (or, $\mathbf{w}_{-1} = \overline{\mathbf{w}}$, if $\overline{\mathbf{w}} \neq \mathbf{0}$) and $P_{-1} = \Pi^{-1}$ and $\xi(-1) = 0$. Then for data $\{\mathbf{u}_i, d(i)\}$ for $i = 0, 1, \ldots$, iterate the following:

$$\begin{aligned}
\mathbf{g}_i &= \lambda^{-1} P_{i-1}\mathbf{u}_i\gamma(i) && \text{gain vector } M \times 1\\
\gamma(i) &= \frac{1}{1 + \lambda^{-1}\mathbf{u}_i^H P_{-1}\mathbf{u}_i} && \text{conversion factor, real scalar}\\
e(i) &= d(i) - \mathbf{u}_i^H \mathbf{w}_{i-1} && \textit{a priori} \text{ error, scalar}\\
\mathbf{w}_i &= \mathbf{w}_{i-1} + \mathbf{g}_i e(i) && \text{updated filter weights } M \times 1\\
P_i &= \lambda^{-1}P_{i-1} - \mathbf{g}_i\mathbf{g}_i^H/\gamma(i) && \text{inverse correlation } M \times M\\
\xi(i) &= \lambda\xi(i-1) + \gamma(i)|e(i)|^2 && \text{Minimum cost, real scalar}
\end{aligned}$$

Due to the structure of the matrix inversion lemma,, the following relations hold at each time:

$$\mathbf{g}_i = P_i\mathbf{u}_i \qquad \gamma(i) = 1 - \mathbf{u}_i^H P_i \mathbf{u}_i = 1 - \mathbf{u}_i^H \mathbf{g}_i \qquad r(i) = \gamma(i)e(i).$$

4 The Inverse QR algorithm for the RLS adaptive filter

There are several QR-related approaches to the RLS adaptive filter. We present one of these to set the stage for the fast algorithm.

Recall the RLS adaptive filter as we derived it previously:

$$\begin{aligned}
\gamma(i) &= \frac{1}{1 + \lambda^{-1}\mathbf{u}_i^H P_{-1}\mathbf{u}_i} && \text{conversion factor, real scalar}\\
\mathbf{g}_i &= \lambda^{-1} P_{i-1}\mathbf{u}_i\gamma(i) && \text{gain vector} M \times 1\\
e(i) &= d(i) - \mathbf{u}_i^H \mathbf{w}_{i-1} && \textit{a priori} \text{ error, scalar}\\
\mathbf{w}_i &= \mathbf{w}_{i-1} + \mathbf{g}_i e(i) && \text{updated filter weights } M \times 1\\
P_i &= \lambda^{-1}P_{i-1} - \mathbf{g}_i\mathbf{g}_I^H/\gamma(i) && \text{Inverse correlation } M \times M\\
\xi(i) &= \lambda\xi(i-1) + \gamma(i)|e(i)|^2 && \text{Minimum cost, real scalar}
\end{aligned}$$

We can write the equations for the the conversion factor and the gain vector as

$$\gamma(i)^{-1} = 1 + \lambda^{-1}\mathbf{u}_i^H P_{-1}\mathbf{u}_i$$

$$\mathbf{g}_i\gamma^{-1}(i) = \lambda^{-1}P_{i-1}\mathbf{u}_i.$$

Let P_{i-1} have the Cholesky factorization

$$P_{i-1} = P_{i-1}^{H/2} P_{i-1}^{1/2}$$

where $P_{i-1}^{1/2}$ is an lower triangular matrix. Now write the conversion factor and gain vector equations as

$$\underbrace{\gamma^{-1/2}(i)\gamma^{-1/2}(i)}_{C^HC} = \underbrace{1}_{A^HA} + \underbrace{\lambda^{-1/2}\mathbf{u}_i^H P_{i-1}^{H/2} P_{i-1}^{1/2}\mathbf{u}_i\lambda^{-1/2}}_{B^HB} \tag{8}$$

$$\underbrace{\gamma^{-1/2}(i)\mathbf{g}_i\gamma^{-1/2}(i)}_{F^HC} = \underbrace{0}_{A^HD} + \underbrace{\lambda^{-1/2}P_{i-1}^{H/2} P_{i-1}^{1/2}\mathbf{u}_i\lambda^{-1/2}}_{E^HB}. \tag{9}$$

Now, comparing with (1), we make the following identifications:

$$C \leftarrow \gamma^{-1/}(i) \qquad A \leftarrow 1 \qquad B \leftarrow \lambda^{-1/2}P_{i-1}^{1/2}\mathbf{u}_i$$

$$F \leftarrow \gamma^{-1/2}(i)\mathbf{g}_i^H \qquad D \leftarrow \mathbf{0} \text{ (row vector)} \qquad E \leftarrow \lambda^{-1/2}P_{i-1}^{H/2}$$

That is, (8) corresponds to the following norm preservation formula:

$$\begin{bmatrix}\gamma^{-1/2} & 0\end{bmatrix}\begin{bmatrix}\gamma^{-1/2}\\ 0\end{bmatrix} = \begin{bmatrix}1 & \lambda^{-1/2}\mathbf{u}_i^H P_{i-1}^{H/2}\end{bmatrix}\begin{bmatrix}1\\ \lambda^{-1/2}P_{i-1}^{1/2}\mathbf{u}_i\end{bmatrix}$$

and (9) corresponds to inner product preservation

$$\begin{bmatrix}\mathbf{g}_i\gamma^{-1/2}(i) & 0\end{bmatrix}\begin{bmatrix}\gamma^{-1/2}(i)\\ 0\end{bmatrix} = \begin{bmatrix}0 & \lambda^{-1/2}P_{i-1}^{H/2}\end{bmatrix}\begin{bmatrix}1\\ \lambda^{-1/2}P_{i-1}^{1/2}\mathbf{u}_i\end{bmatrix}$$

Motivated by the problem solved before, we form the pre-array

$$\mathcal{A} = \begin{bmatrix}A & D\\ B & E\end{bmatrix} = \begin{bmatrix}1 & \mathbf{0}\\ \lambda^{-1/2}P_{i-1}^{1/2}\mathbf{u}_i & \lambda^{-1/2}P_{i-1}^{H/2}\end{bmatrix}. \tag{10}$$

For example, for $M = 3$, the pre-array has the form

$$\mathcal{A} = \left[\begin{array}{c|ccc} 1 & 0 & 0 & 0\\ \hline \times & \times & \times & \times\\ \times & 0 & \times & \times\\ \times & 0 & 0 & \times \end{array}\right]$$

We now seek a unitary matrix Θ which transforms this to a lower triangular form

$$\mathcal{B} = \Theta\mathcal{A} = \begin{bmatrix}\times & \mathbf{y}^H\\ \mathbf{0} & \mathsf{z}\end{bmatrix}.$$

(That is, $\mathbf{y}$ is a column vector, and $\mathbf{y}^H$ is a row vector.) The post-array has the form

$$\mathcal{B} = \left[\begin{array}{c|ccc} \times & \times & \times & \times\\ \hline 0 & \times & \times & \times\\ 0 & 0 & \times & \times\\ 0 & 0 & 0 & \times \end{array}\right]$$

We identify

$$\times = \gamma^{-1/2}(i)$$

and

$$\mathbf{y} = \mathbf{g}_i\gamma^{-1/2}.$$

This can also be seen by squaring both sides and equating elements:

$$\mathcal{A}^H\mathcal{A} = \begin{bmatrix}1 & \lambda^{-1/2}\mathbf{u}_i^H P_{i-1}^{H/2}\\ \mathbf{0} & \lambda^{-1/2}P_{i-1}^{H/2}\end{bmatrix}\begin{bmatrix}1 & \mathbf{0}\\ \lambda^{-1/2}P_{i-1}^{1/2}\mathbf{u}_i & \lambda^{-1/2}P_{i-1}^{1/2}\end{bmatrix} = \begin{bmatrix}1+\lambda^{-1}\mathbf{u}_i^H P_{i-1}\mathbf{u}_i & \lambda^{-1}\mathbf{u}_i^H P_{i-1}\\ \lambda^{-1}P_{i-1}\mathbf{u}_i & \lambda^{-1}P_{i-1}\end{bmatrix}$$

$$= \mathcal{B}^H\mathcal{B} = \begin{bmatrix}\times^H & \mathbf{0}\\ \mathbf{y} & \mathsf{z}\end{bmatrix}\begin{bmatrix}\times & \mathbf{y}^H\\ \mathbf{0} & \mathsf{z}^H\end{bmatrix} = \begin{bmatrix}|\times|^2 & \times^H\mathbf{y}^H\\ \mathbf{y}\times & \mathbf{y}\mathbf{y}^H + \mathsf{z}^H\end{bmatrix}$$

from which

$$\begin{aligned} |\mathsf{x}|^2 &= 1 + \lambda^{-1}\mathbf{u}_i^H P_{i-1}\mathbf{u}_i \\ \mathsf{x}\mathbf{y} &= \lambda^{-1}P_{i-1}\mathbf{u}_i \\ \mathbf{y}\mathbf{y}^H + \mathsf{z}^H &= \lambda^{-1}P_{i-1} \end{aligned}$$

The Inverse QR RLS algorithm can be summarized as follows:

1. From the data available at time $i-1$, form the pre-array $\mathcal{A}$ as in (10).
2. Find a unitary Θ such that $\mathcal{B} = \Theta\mathcal{A}$ is upper triangular, and identify its elements as

$$\mathcal{B} = \Theta\mathcal{A} = \begin{bmatrix} \gamma^{-1/2}(i) & \mathbf{g}_i^H\gamma^{-1/2}(i) \\ \mathbf{0} & \lambda^{-1/2}P_i^{1/2} \end{bmatrix}.$$

3. Update the weight vector as

$$\mathbf{w}_i = \mathbf{w}_{i-1} + \mathbf{g}_i(d(i) - \mathbf{u}_i^H\mathbf{w}_{i-1}).$$

The number of computations is $O(M^2)$ per iteration.

Note that in this algorithm, we never explicity compute, or need to compute P_i. Instead, we propagate only a *square root* of the matrix. This avoids some potential numerical problems (not squaring a matrix), and ensures that the P_i associated with the algorithm is, in fact, positive definite. (We know we can compute a square root of P_i, since we already have the square root on hand.)

This algorithm is also known as the square-root RLS, since it propagates the square-root factor $P_i^{1/2}$.

We note that reducing the pre-array to the post-array is essentially the same as the QR factorization of the pre-array (we obtain the transpose of the Q matrix in the usual QR factorization).

5 The FARLS algorithm

With all this as background, we are now ready to look at a faster algorithm for the RLS. The RLS algorithms described above assume a general form for the regressors $\mathbf{u}_i$. However, in many cases these have a shift structure, and we will be exploiting this. The Fast Array Algorithm is an RLS algorithm that has linear complexity on temporal adaptive filtering problems.

The filter uses an input vector of length M formed from the input $u(i)$ by

$$\mathbf{u}_i = \begin{bmatrix} u(i) & u(i-1) & \cdots & u_{i-M+1} \end{bmatrix}^T$$

We restate the RLS algorithm, starting from an initial $\mathbf{w}_{-1} = \overline{\mathbf{w}}$ (the initial weight vector) and $P_{-1} = \Pi^{-1}$ (the initial inverse covariance) and $\xi(-1) = 0$, repeat for $i = 0, 1, 2, \ldots$

$$\begin{aligned} \gamma(i) &= \frac{1}{1 + \lambda^{-1}u_i^H P_{i-1}\mathbf{u}_i} \qquad \text{scalar} \\ \mathbf{g}_i &= \lambda^{-1}\gamma(i)P_{i-1}\mathbf{u}_i = P_i\mathbf{u}_i \qquad \text{gain vector; column vector} \\ e(i) &= d(i) - \mathbf{u}_i^H\mathbf{w}_{i-1} \qquad \text{prior error; scalar} \\ \mathbf{w}_i &= \mathbf{w}_{i-1} + \mathbf{g}_i e(i) \qquad \text{weight vector update; column vector} \\ P_i &= \lambda^{-1}P_{i-1} - \mathbf{g}_i\mathbf{g}_i^H/\gamma(i) \qquad \text{inv. cov. update; matrix} \\ r(i) &= d(i) - \mathbf{u}_i^H\mathbf{w}_i \qquad \text{posterior error; scalar} \\ \xi(i) &= \lambda\xi(i-1) + r(i)e^*(i) \qquad \text{scalar} \end{aligned} \tag{11}$$

The difficulty in finding an $O(M)$ algorithm is apparent: even if computing the elements of P_i was linear, simply computing all of them is $O(M^2)$ complexity. The crux of the issue, then, is that we cannot afford to compute all of the elements of P_i. However, because of the shifting nature of the $\mathbf{u}_i$, it turns out that there is redundancy in the P_i that can be exploited. We will find that there is *shift structure* to the problem we can use to our benefit.

An obvious, but critical observation is that since

$$\mathbf{u}_{i-1} = \begin{bmatrix} u(i-1) \\ u(i-2) \\ \cdots \\ u(i-M) \end{bmatrix},$$

it follows that

$$\begin{bmatrix} u(i) \\ \mathbf{u}_{i-1} \end{bmatrix} = \begin{bmatrix} \mathbf{u}_i \\ u(i-M) \end{bmatrix}.$$

This shift structure property is the source of the processing efficiency.

The update for $\mathbf{g}_i$ for two time instants yields

$$\mathbf{g}_i\gamma^{-1}(i) = \lambda^{-1}P_{i-1}\mathbf{u}_i \qquad \mathbf{g}_{i-1}\gamma^{-1}(i-1) = \lambda^{-1}P_{i-2}\mathbf{u}_{i-1}. \tag{12}$$

Let us express the "extended" vector

$$\begin{bmatrix} P_{i-1}\mathbf{u}_i \\ 0 \end{bmatrix}$$

in the two equivalent forms

$$\begin{bmatrix} P_{i-1}\mathbf{u}_i \\ 0 \end{bmatrix} = \begin{bmatrix} P_{i-1} & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} \mathbf{u}_i \\ u(i-M) \end{bmatrix} = \begin{bmatrix} P_{i-1} & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} u(i) \\ \mathbf{u}_{i-1} \end{bmatrix}. \tag{13}$$

Let us also express the extended vector

$$\begin{bmatrix} 0 \\ P_{i-2}\mathbf{u}_{i-1} \end{bmatrix}$$

in the form

$$\begin{bmatrix} 0 \\ P_{i-2}\mathbf{u}_{i-1} \end{bmatrix} = \begin{bmatrix} 0 & 0 \\ 0 & P_{i-2} \end{bmatrix} \begin{bmatrix} u(i) \\ \mathbf{u}_{i-1} \end{bmatrix} \tag{14}$$

Scaling and subtracting (14) from (13) we obtain

$$\lambda^{-1}\begin{bmatrix} P_{i-1}\mathbf{u}_i \\ 0 \end{bmatrix} - \lambda^{-1}\begin{bmatrix} 0 \\ P_{i-2}\mathbf{u}_{i-1} \end{bmatrix} = \lambda^{-1}\left(\begin{bmatrix} P_{i-1} & 0 \\ 0 & 0 \end{bmatrix} - \begin{bmatrix} 0 & 0 \\ 0 & P_{i-2} \end{bmatrix}\right)\begin{bmatrix} u(i) \\ \mathbf{u}_{i-1} \end{bmatrix}.$$

Let δP_{i-1} denote the difference

$$\delta P_{i-1} = \begin{bmatrix} P_{i-1} & 0 \\ 0 & 0 \end{bmatrix} - \begin{bmatrix} 0 & 0 \\ 0 & P_{i-2} \end{bmatrix}$$

so we can write

$$\begin{bmatrix} P_{i-1}\mathbf{u}_i \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ P_{i-2}\mathbf{u}_{i-1} \end{bmatrix} + \delta P_{i-1}\begin{bmatrix} u(i) \\ \mathbf{u}_{i-1} \end{bmatrix}.$$

Now (12) can be expressed as

$$\begin{bmatrix} \mathbf{g}_i\gamma^{-1}(i) \\ 0 \end{bmatrix} = \begin{bmatrix} \lambda^{-1}P_{i-1}\mathbf{u}_i \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ \lambda^{-1}P_{i-2}\mathbf{u}_{i-1} \end{bmatrix} + \lambda^{-1}\delta P_{i-1}\begin{bmatrix} u(i) \\ \mathbf{u}_{i-1} \end{bmatrix} = \begin{bmatrix} 0 \\ \mathbf{g}_{i-1}\gamma^{-1}(i-1) \end{bmatrix} + \lambda^{-1}\delta P_{i-1}\begin{bmatrix} u(i) \\ \mathbf{u}_{i-1} \end{bmatrix}.$$

We thus observe that to find the update $\mathbf{g}_i\gamma^{-1}(i)$, it is only necessary to know the difference δP_{i-1}, not the full matrices P_i and P_{i-1}.

In reality we need $\mathbf{g}_i$, not the product $\mathbf{g}_i\gamma^{-1}(i)$. We thus need a way to find $\gamma(i)$ (or $\gamma^{-1}(i)$) that can also be efficiently computed. From (11) we have

$$\gamma^{-1}(i) = 1 + \lambda^{-1}\mathbf{u}_i^H P_{i-1}\mathbf{u}_i \qquad \gamma^{-1}(i-1) = 1 + \lambda^{-1}\mathbf{u}_{i-1}^H P_{i-2}\mathbf{u}_{i-1}$$

so that, by subtraction we obtain

$$\gamma^{-1}(i) - \gamma^{-1}(i-1) = \lambda^{-1}[\mathbf{u}_i^H P_{i-1}\mathbf{u}_i - \mathbf{u}_{i-1}^H P_{i-2}\mathbf{u}_{i-2}].$$

Let us denote the difference here as

$$\nabla = \mathbf{u}_i^H P_{i-1} \mathbf{u}_i - \mathbf{u}_{i-1}^H P_{i-2} \mathbf{u}_{i-2}.$$

Again using the shift structure we can write

$$\nabla = \begin{bmatrix} u(i)^H & \mathbf{u}_{i-1}^H \end{bmatrix} \left(\begin{bmatrix} P_{i-1} & 0 \\ 0 & 0 \end{bmatrix} - \begin{bmatrix} 0 & 0 \\ 0 & P_{i-2} \end{bmatrix} \right) \begin{bmatrix} u(i) \\ \mathbf{u}_{i-1} \end{bmatrix} = \begin{bmatrix} u(i)^H & \mathbf{u}_{i-1}^H \end{bmatrix} \delta P_{i-1} \begin{bmatrix} u(i) \\ \mathbf{u}_{i-1} \end{bmatrix}$$

We thus can express the update for $\gamma^{-1}(i)$ using a quantity that depends only on δP_{i-1}, and not on P_i or P_{i-1}:

$$\gamma^{-1}(i) = \gamma^{-1}(i-1) + \lambda^{-1} \begin{bmatrix} u(i)^* & \mathbf{u}_{i-1}^H \end{bmatrix} \delta P_{i-1} \begin{bmatrix} u(i) \\ \mathbf{u}_{i-1} \end{bmatrix}$$

Now we set up some initial conditions that will make an efficient representation for the δP_{i-1}. Let us assume that prior inputs are 0: $u(i) = 0$ for $i < 0$, so that $\mathbf{u}_{-1} = \mathbf{0}$. Set the initial conditions

$$\mathbf{g}_{-1} = \mathbf{0} \qquad \gamma(-1) = 1 \qquad P_{-1} = \Pi^{-1} \qquad P_{-2} = \lambda P_{-1} = \lambda \Pi^{-1}$$

so that

$$\delta P_{-1} = \begin{bmatrix} \Pi^{-1} & 0 \\ 0 & 0 \end{bmatrix} - \begin{bmatrix} 0 & 0 \\ 0 & \lambda \Pi^{-1} \end{bmatrix}$$

If we now choose Π to have the form

$$\Pi = \eta \, \mathrm{diag}(\lambda^2, \lambda^3, \ldots, \lambda^{M+1})$$

then most of the elements cancel and we obtain

$$\delta P_{-1} = \eta \lambda^2 \, \mathrm{diag}(1, 0, \ldots, 0, -\lambda^M)$$

Here, η is a (usually large) positive scalar constant. Observe that δP_{-1} is a rank-2 matrix which we can factor as

$$\delta P_{-1} = \lambda L_{-1} S_{-1} L_{-1}^H$$

where

$$S_{-1} = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

is a *signature matrix* and

$$L_{-1} = \sqrt{\eta \lambda} \begin{bmatrix} 1 & 0 \\ 0 & 0 \\ \vdots & \vdots \\ 0 & 0 \\ 0 & \lambda^{M/2} \end{bmatrix} \qquad ((M+1) \times 2).$$

It can be shown that δP_i should preserve the rank-2 structure for all i. If the rank ever changes, that is an indication that something has gone wrong — instability has set in already.

To recap, using these particular initial conditions, and assuming the preservation of the rank structure at each time of δP_i, we have

$$\gamma^{-1}(i) = \gamma^{-1}(i-1) + \begin{bmatrix} u(i)^H & \mathbf{u}_{i-1}^H \end{bmatrix} L_{i-1} S_{i-1} L_{i-1}^H \begin{bmatrix} u(i) \\ \mathbf{u}_{i-1} \end{bmatrix}$$

$$\begin{bmatrix} \mathbf{g}_i \gamma^{-1}(i) \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ \mathbf{g}_{i-1} \gamma^{-1}(i-1) \end{bmatrix} + L_{i-1} S_{i-1} L_{i-1}^H \begin{bmatrix} u_i \\ \mathbf{u}_{i-1} \end{bmatrix}.$$

or, equivalently

$$\begin{aligned}
\underbrace{\gamma^{-1/2}(i)\gamma^{-1/2}(i)}_{C^HC} &= \underbrace{\gamma^{-1/2}(i-1)\gamma^{-1/2}(i-1)}_{A^HA} + \underbrace{\begin{bmatrix} u(i)^H & \mathbf{u}_{i-1}^H \end{bmatrix} L_{i-1}S_{i-1}L_{i-1}^H \begin{bmatrix} u(i) \\ \mathbf{u}_{i-1} \end{bmatrix}}_{B^HSB} \\
\underbrace{\begin{bmatrix} \mathbf{g}_i\gamma^{-1/2}(i) \\ 0 \end{bmatrix}\gamma^{-1/2}(i)}_{F^HC} &= \underbrace{\begin{bmatrix} 0 \\ \mathbf{g}_{i-1}\gamma^{-1/2}(i-1) \end{bmatrix}\gamma^{-1/2}(i-1)}_{D^HA} + \underbrace{L_{i-1}S_{i-1}L_{i-1}^H \begin{bmatrix} u_i \\ \mathbf{u}_{i-1} \end{bmatrix}}_{E^HSB}.
\end{aligned} \tag{15}$$

where we have made the identifications

$$C = \gamma^{-1/2}(i) \qquad A = \gamma^{-1/2}(i-1) \qquad B = L_{i-1}^H \begin{bmatrix} u(i) \\ \mathbf{u}_{i-1} \end{bmatrix}$$

$$F^H = \begin{bmatrix} \mathbf{g}_i\gamma^{-1/2}(i) \\ 0 \end{bmatrix} \qquad D^H = \begin{bmatrix} 0 \\ \mathbf{g}_{i-1}\gamma^{-1/2}(i-1) \end{bmatrix} \qquad E^H = L_{i-1},$$

the equations in (15) can be expressed as

$$\begin{aligned}
C^HC &= A^HA + B^HSB \\
F^HC &= D^HA + E^HSB
\end{aligned} \tag{16}$$

In these equations, A, B, D, E, and S are known, and the problem is to solve for C and F, and to do so in a way that is computationally stable and fast. These are similar to the equations in (1), except there is an extra factor of S present in some of the terms. This factor of S introduces significant changes to the way the problem is solved.

Clearly there are algebraic solutions to (16). However, note that there is a quadratic in C; as a general rule in linear numerical methods, computing quadratic quantities such as CC^* should be avoided. (This follows from the rule of thumb that if X is an ill-conditioned matrix, then XX^T has even worse condition number.) Also, straightforward algebra would require more computations than the approach to be developed here.

The solution will be obtained by using hyperbolic rotations. These are more stable than more direct approaches might be, and also computationally efficient. But they are not stable enough to avoid all numerical problems.

5.1 J-Unitary matrices

In order to deal with the more general form in (16), it is necessary to introduce the concept of J-unitary matrices as a generalization of the idea of unitary matrices.

Recall that a unitary matrix Q is such that $Q^HQ = I$ and $QQ^H = I$. A generalization of this concept is a J-unitary matrix. A J-unitary matrix Θ is such that

$$\Theta J \Theta^H = J$$

and

$$\Theta^H J \Theta = J,$$

where J is a diagonal matrix of ± 1 entries. Such a matrix is sometimes called a *signature matrix*. When $J = I$ (the identity), then we have the special case of unitary matrices.

Just as we can construct a unitary matrix using a series of Givens rotations or Householder transformations, we can construct a J-unitary matrix out of suitably modified operations, called hyperbolic Givens rotations and hyperbolic Householder transformations.

5.2 Array algorithm

We will present some details of how to compute an appropriate J-unitary matrix below. For now, let us assume that we have the ability to form J-unitary matrices as needed.

To solve (16), we form the matrix

$$\mathcal{A} = \begin{bmatrix} A & D \\ B & E \end{bmatrix}$$

To be more explicit, we denote the sizes of the block elements of $\mathcal{A}$ as subscripts:

$$\mathcal{A} = \begin{bmatrix} A_{1\times 1} & D_{1\times(M+1)} \\ B_{2\times 1} & E_{2\times(M+1)} \end{bmatrix}$$

We will seek a matrix Θ such that

$$\Theta\mathcal{A} = \begin{bmatrix} X & Y \\ \mathbf{0} & Z \end{bmatrix} \tag{17}$$

where the $\mathbf{0}$ element is 2×1. The matrix Θ must be $I \oplus S$-unitary, such that

$$\Theta^H \begin{bmatrix} I & \\ & S \end{bmatrix} \Theta = \begin{bmatrix} I & \\ & S \end{bmatrix}.$$

Finding such a matrix Θ is discussed below. For now, assuming that we have such a matrix having the transformation (17), then "squaring" the equation to form $\mathcal{A}^H\Theta^H \begin{bmatrix} I & \\ & S \end{bmatrix} \Theta\mathcal{A}$, we obtain

$$\begin{bmatrix} A & D \\ B & E \end{bmatrix}^H \Theta^H \begin{bmatrix} I & \\ & S \end{bmatrix} \Theta \begin{bmatrix} A & D \\ B & E \end{bmatrix} = \begin{bmatrix} X & Y \\ \mathbf{0} & Z \end{bmatrix}^H \begin{bmatrix} I & \\ & S \end{bmatrix} \begin{bmatrix} X & Y \\ \mathbf{0} & Z \end{bmatrix}$$

By the $I \oplus S$-unitary property we can re-write the product on the left as

$$\begin{bmatrix} A & D \\ B & E \end{bmatrix}^H \begin{bmatrix} I & \\ & S \end{bmatrix} \begin{bmatrix} A & D \\ B & E \end{bmatrix} = \begin{bmatrix} X & Y \\ \mathbf{0} & Z \end{bmatrix}^H \begin{bmatrix} I & \\ & S \end{bmatrix} \begin{bmatrix} X & Y \\ \mathbf{0} & Z \end{bmatrix}$$

Expanding out the products on both sides we obtain

$$\begin{bmatrix} A^HA + B^HSB & A^HD + B^HSE \\ D^HA + E^HSB & D^HD + E^HSE \end{bmatrix} = \begin{bmatrix} X^HX & X^HY \\ Y^HX & Y^HY + Z^HSZ \end{bmatrix}.$$

Comparing block by block, in light of (16) we see that we can identify

$$X = C = \gamma^{-1/2}(i) \qquad \text{and} \qquad Y = F = \begin{bmatrix} \mathbf{g}_i\gamma^{-1/2}(i) \\ 0 \end{bmatrix}^H.$$

That is, if we can find the matrix Θ (with the appropriate properties, and using an algorithm with sufficient numerical accuracy) and compute $\Theta\mathcal{A}$, then we immediately obtain our desired values C and F. Finding such a Θ is the heart of the matter computationally. However, to see how the algorithm proceeds, we will assume that Θ is known, and see how to put the algorithm together.

From the lower right block we identify the equation

$$\begin{aligned} Z^HSZ &= D^HD + E^HSE - Y^HY \\ &= \begin{bmatrix} 0 \\ \mathbf{g}_{i-1}\gamma^{-1/2}(i-1) \end{bmatrix} \begin{bmatrix} 0 \\ \mathbf{g}_{i-1}\gamma^{-1/2}(i-1) \end{bmatrix}^H + L_{i-1}SL_{i-1}^H - \begin{bmatrix} \mathbf{g}_i\gamma^{-1/2}(i) \\ 0 \end{bmatrix} \begin{bmatrix} \mathbf{g}_i\gamma^{-1/2}(i) \\ 0 \end{bmatrix}^H \\ &= \begin{bmatrix} 0 \\ \mathbf{g}_{i-1}\gamma^{-1/2}(i-1) \end{bmatrix} \begin{bmatrix} 0 \\ \mathbf{g}_{i-1}\gamma^{-1/2}(i-1) \end{bmatrix}^H + \left(\begin{bmatrix} \lambda^{-1}P_{i-1} & 0 \\ 0 & 0 \end{bmatrix} - \begin{bmatrix} 0 & 0 \\ 0 & \lambda^{-1}P_{i-2} \end{bmatrix} \right) - \\ &\qquad - \begin{bmatrix} \mathbf{g}_i\gamma^{-1/2}(i) \\ 0 \end{bmatrix} \begin{bmatrix} \mathbf{g}_i\gamma^{-1/2}(i) \\ 0 \end{bmatrix}^H \end{aligned}$$

from the definition of δP_{i-1}. Now recall from (11) that

$$P_i = \lambda^{-1} P_{i-1} - \mathbf{g}_i \mathbf{g}_i^H / \gamma(i)$$

so that

$$\mathbf{g}_i \mathbf{g}_i^H / \gamma(i) = P_i - \lambda^{-1} P_{i-1}.$$

Using this in the last equation (in two places at i and $i-1$) we have

$$\begin{aligned} ZSZ^* &= \begin{bmatrix} 0 & 0 \\ 0 & \lambda^{-1}P_{i-2} - P_{i-1} \end{bmatrix} + \begin{bmatrix} \lambda^{-1}P_{i-1} & 0 \\ 0 & 0 \end{bmatrix} - \begin{bmatrix} 0 & 0 \\ 0 & \lambda^{-1}P_{i-2} \end{bmatrix} - \begin{bmatrix} \lambda^{-1}P_{i-1} - P_i & 0 \\ 0 & 0 \end{bmatrix} \\ &= \begin{bmatrix} P_i & 0 \\ 0 & 0 \end{bmatrix} - \begin{bmatrix} 0 & 0 \\ 0 & P_{i-1} \end{bmatrix} = \delta P_i \end{aligned}$$

But by the structure of δP_i we can write $\delta P_i = \lambda L_i S L_i^H$, so that

$$Z^H S Z = \lambda L_i S L_i^H$$

We thus identify $Z = \sqrt{\lambda} L_i^H$. This enables us to go from one step to the next.

5.3 RLS Array Algorithm

At each step of the algorithm, form $\mathcal{A} = \begin{bmatrix} A & D \\ B & E \end{bmatrix}$, or, more explicitly

$$\mathcal{A} = \begin{bmatrix} \gamma^{-1/2}(i) & \begin{bmatrix} 0 & \mathbf{g}_{i-1}\gamma^{-1/2}(i-1) \end{bmatrix} \\ L_{i-1}^H \begin{bmatrix} u(i) \\ \mathbf{u}_{i-1} \end{bmatrix} & L_{i-1}^H \end{bmatrix}$$

This matrix is called the pre-array. Now find a 3×3 matrix Θ_i which annihilates the last two entries in the first column of $\mathcal{A}$. Form

$$\Theta_i \mathcal{A} = \mathcal{B}$$

as

$$\Theta_i \begin{bmatrix} \gamma^{-1/2}(i) & \begin{bmatrix} 0 & \mathbf{g}_{i-1}\gamma^{-1/2}(i-1) \end{bmatrix} \\ L_{i-1}^H \begin{bmatrix} u(i) \\ \mathbf{u}_{i-1} \end{bmatrix} & L_{i-1}^H \end{bmatrix} = \begin{bmatrix} \gamma^{-1/2}(i) & \begin{bmatrix} \mathbf{g}_i\gamma^{-1/2} & 0 \end{bmatrix} \\ \begin{bmatrix} 0 \\ \mathbf{0} \end{bmatrix} & \sqrt{\lambda} L_i^H \end{bmatrix}$$

Identify from $\mathcal{B}$ the elements

$$C = \gamma^{-1/2}(i) \qquad \text{and} \qquad F^H = \begin{bmatrix} \mathbf{g}_i \gamma^{-1/2}(i) \\ 0 \end{bmatrix}$$

Now the weights are updated using these identified values as

$$\mathbf{w}_i = \mathbf{w}_{i-1} + \left[\mathbf{g}_i \gamma^{-1/2}(i)\right] \left[\gamma^{-1/2}(i)\right]^{-1} \left[d(i) - \mathbf{u}_i \mathbf{w}_{i-1}\right]$$

5.4 Finding Θ

The goal of Θ is to annihilate two elements of the first column of $\mathcal{A}$. Let the first column denoted as

$$\begin{bmatrix} a \\ b \\ c \end{bmatrix}.$$

Then we desire a matrix Θ such that

$$\Theta \begin{bmatrix} a \\ b \\ c \end{bmatrix} = \begin{bmatrix} e \\ 0 \\ 0 \end{bmatrix}.$$

Furthermore, the matrix has to have the property that

$$\Theta^H \begin{bmatrix} 1 & \\ & S \end{bmatrix} \Theta = \begin{bmatrix} 1 & \\ & S \end{bmatrix}.$$

We will accomplish this in two steps. First, we find a 2×2 matrix Θ_1 which annihilates the first two elements:

$$\Theta_1 \begin{bmatrix} a & b \end{bmatrix} = \begin{bmatrix} d & 0 \end{bmatrix}$$

with $\Theta_1^H \Theta_1 = I$. That is, Θ_1 is a *unitary matrix*. Then we find a 2×2 matrix which annihilates the first and third elements:

$$\Theta_2 \begin{bmatrix} b \\ c \end{bmatrix} = \begin{bmatrix} e \\ 0 \end{bmatrix}$$

such that $\Theta_2^H J \Theta_2 = J$, where $J = \begin{bmatrix} 1 & \\ & -1 \end{bmatrix}$. These two operations can be combined as

$$\Theta = \begin{bmatrix} \Theta_{2,1,1} & 0 & \Theta_{2,1,2} \\ 0 & 1 & 0 \\ \Theta_{2,2,1} & 0 & \Theta_{2,2,2} \end{bmatrix} \begin{bmatrix} \Theta_{1,1,1} & \Theta_{1,1,2} & 0 \\ \Theta_{1,2,1} & \Theta_{1,2,2} & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Finding Θ_1 is straightforward and numerically stable. It can be accomplished using Givens rotations. Finding Θ_2 is straightforward, but not so numerically stable.

To reiterate, we want to find Θ_2 such that

$$\Theta_2 \begin{bmatrix} b \\ c \end{bmatrix} = \begin{bmatrix} e \\ 0 \end{bmatrix}$$

and $\Theta_2^H J \Theta_2 = J$. We see that

$$\begin{bmatrix} b^* & c^* \end{bmatrix} \Theta^H J \Theta \begin{bmatrix} b \\ c \end{bmatrix} = \begin{bmatrix} e^* & 0 \end{bmatrix} J \begin{bmatrix} e \\ 0 \end{bmatrix}$$

so that

$$|b|^2 - |c|^2 = |e|^2$$

Since $|e|^2 > 0$, we must have $|b| > |c|$. (Failure of this condition is one indication that the RLS algorithm has gone unstable.)

This problem can be understood geometrically as possible. Consider a hyperbola $x^2 - y^2 = e^2$, as shown in figure 1. When $x = b$ and $y = c$, we obtain a point on the hyperbola. The J-unitary transformation yields a new point $\Theta_2 \left[\begin{smallmatrix} b \\ s \end{smallmatrix} \right]$ which also lies on this hyperbola: we are essentially "rotating" (in some kind of hyperbolic sense) until we arrive at the axis.

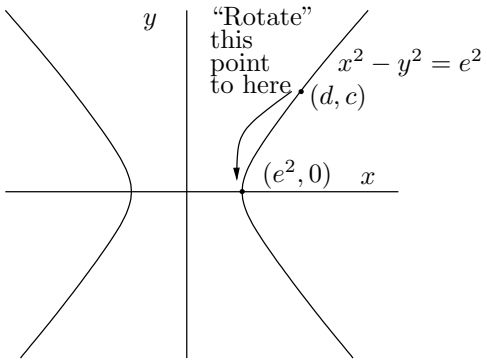


Figure 1: Hyperbola showing domain of J-unitary rotation

A J-unitary matrix which achieves this is

$$\Theta_2 = \frac{1}{\sqrt{1-\rho^2}} \begin{bmatrix} 1 & -\rho \\ -\rho & 1 \end{bmatrix} \qquad \text{where} \qquad \rho = \frac{d}{c}.$$

This form is called the *Hyperbolic Givens rotation*, being a generalization of the conventional circular Givens rotation. Note that when $d \approx \pm c$ — that is, way out on the hyperbola so that the asymptotic lines are approached — then Θ_2 is poorly conditioned, being proportional to something nearly

$$\begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} \quad \text{or} \quad \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$$

(with a very large scale factor in front). This poor conditioning is, in fact, the source of the instability in the RLS algorithm.

While fairly stable, the numerics of this transformation can be improved. Write

$$\Theta_2 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ -1 & 1 \end{bmatrix} \begin{bmatrix} \sqrt{\frac{d+c}{d-c}} & 0 \\ 0 & \sqrt{\frac{d-c}{d+c}} \end{bmatrix} \begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix} \frac{1}{\sqrt{2}}.$$

This is the eigendecomposition of Θ_2. It is claimed to be numerically stable, but also has trouble.

An alternative method of finding Θ_2 is using a hyperbolic Householder transformation. Let $\mathbf{z} = \begin{bmatrix} d & c \end{bmatrix}$. We compute $\alpha = \pm\sqrt{\mathbf{z}J\mathbf{z}^T}$ then define $\mathbf{g} = \mathbf{z} - \alpha e_0$, where $e_0 = \begin{bmatrix} 1 & 0 \end{bmatrix}$. We form the matrix

$$\Theta_2 = I - 2J\frac{\mathbf{g}^T\mathbf{g}}{\mathbf{g}J\mathbf{g}^T}. \tag{18}$$

It can be shown that this does the appropriate transformation.

5.5 A C language implementation

An efficient C-language implementation of this, which avoids multiple loops, is shown below. This actually deals with J-unitary operations on the right, but follows the principles outlined above.

```
gammainvsq = 1;
e_new = 0;
ux0 = 0; ux1 = 0;
while(cnt && !isnanflag) {          // loop over all the data in file
   for(t=0; t<cnt; t++) {           // loop over data in block
      d = dbuff[t];                 // get new data sample
      ux[0] = xbuff[t];

      ux0 += ux[0]*A[1][1];         // update ux*L
      ux1 += ux[0]*A[1][2];

      // build top row of matrix
      A[0][0] = gammainvsq;  A[0][1] = ux0; A[0][2] = ux1;
      A[1][0] = 0;                  // and second row

      // find a transformation which zeros the two elements on
      // first row of A
      GIVENS(A[0][0], A[0][1], &c, &s);
      APPLYGIVENS(&(A[0][0]),&(A[0][1]), c,s);
      HYPER( A[0][0],A[0][2], &ch, &sh);
      APPLYHYPER(&(A[0][0]),&(A[0][2]), ch, sh);
      gammainvsq_old = gammainvsq;
      gammainvsq = A[0][0];          // save new value
      e_new_old = e_new;             // save the old prior error
      e_new = d;                     // start accumulating new prior error
      ux0 = 0;                       // start accum u*L
      ux1 = 0;
```

```
      ux[M] = ux[M-1];
      // loop over other rows of A, applying transformation and
      // accumulating stuff and shifting for next time
      for(i =M+1; i >= M; i--) {
         w[i-2] += A[i][0]/gammainvsq_old*e_new_old;
         APPLYGIVENS(&(A[i][0]),&(A[i][1]), c,s);
         APPLYHYPER(&(A[i][0]),&(A[i][2]), ch, sh);
         A[i][1] *= lamrtinv;  A[i][2] *= lamrtinv;
      }
      A[M+1][0] = A[M][0];  // slide down in preparation for next time

      // loop over other rows of A, applying transformation and
      // accumulating stuff and shifting for next time
      for(i = M-1; i > 0; i--) {
         ux[i] = ux[i-1];
         w[i-2] += A[i][0]/gammainvsq_old*e_new_old;
                              // does one extra computation
         APPLYGIVENS(&(A[i][0]),&(A[i][1]), c,s);
         APPLYHYPER(&(A[i][0]),&(A[i][2]), ch, sh);

         e_new -= ux[i]*w[i];
         A[i][1] *= lamrtinv;  A[i][2] *= lamrtinv;
         A[i+1][0] = A[i][0]; /* slide down in preparation for next time */
         ux0 += ux[i]*A[i+1][1];
         ux1 += ux[i]*A[i+1][2];
      }
      e_new -= ux[0]*w[0];
      ux0 += ux[M]*A[M+1][1];
      ux1 += ux[M]*A[M+1][2];
      ebuff[t] = (float)e_new;
   }
   fwrite(ebuff, sizeof(float), cnt, fe);
   cnt = fread(dbuff, sizeof(float), BUFF_SIZE, fd);
   cnt = fread(xbuff, sizeof(float), BUFF_SIZE, fx);
}
```